Deliverable 1 Guidance

Sparse Matrix-Vector Multiplication (SpMV) Investigation

GPU Computing 2025-2026 • Deliverable

Goal. Develop and compare SpMV implementations GPU Ampere, over set of sparse matrix formats, and explain the observed performance differences in terms of algorithmic technique and memory behavior, validate results and correctness on CPU-based implementation.

1. Scope and expected outcome

This deliverable focuses on the design of a single-GPU Sparse Matrix Vector Multiplication (SpMV). Students are expected to move beyond simple evaluation of data structures for SpMV (e.g. COO-versus-CSR comparison) and produce a small experimental study on how sparse storage format and implementation strategy affect SpMV performance. The emphasis is not only on obtaining numbers, but on interpreting them convincingly.

You may choose the sparse format(s) you consider most appropriate for your study. COO and CSR are valid options, but they are not the only ones. Alternatives such as ELL, HYB, JDS, DIA, or block-oriented formats may be used when justified by matrix structure [1].
A CPU baseline is mandatory. It does not need to be aggressively optimized for multicore CPUs: a simple sequential implementation or a straightforward OpenMP version is sufficient, provided it is correct and useful for validation only (no performance evaluation is mandatory).
The work should be framed as an investigation. You can implement techniques introduced in State-of-the-art-paper [1,2]. Compare with CuSparse implementation (does it matter if your implementation is slower).
Your report should explain why some formats or kernels perform well on some matrices and poorly on others.
Provide at least two versions. One with a straightforward parallelization (e.g., using a parallelization over the CSR), the other could use techniques and features to reduce the cost of accessing the global memory (using shared memory is fundamental) and improve the load balancing. See for example those reported in [2] and other references listed below.

2. Required deliverables

Artifact	Content	Notes
Code	CPU reference implementation and at least one GPU implementation. Delivered on Git!	The code must read Matrix Market (.mtx) inputs and generate a random dense vector.
Experiments	Timing and FLOPS or GFLOP/s for all tested cases.	Use repeated runs and report how timings were obtained.
Analysis	Explanation of trends across matrices, formats, and kernels.	Pure tables of numbers are not sufficient.
Report	Up to 4 pages, following the course report structure.	Concise, but technically argued. Use pseudocode to describer your algorithms.

3. Dataset selection

Dataset choice is part of the scientific quality of the deliverable. Use the SuiteSparse collection and select 10 matrices that cover different regimes in size and sparsity structure. The set should not be accidental: it should help you stress different access patterns and expose when a format is advantageous.

1. Select 10 matrices from SuiteSparse with diversity in: matrix dimensions, number of nonzeros, average nonzeros per row, row-length variability, and degree of structural regularity. I suggest using the same matrices reported in [2].
2. Include both relatively structured matrices and clearly unstructured matrices. A good portfolio contains matrices with predictable row patterns as well as matrices with highly irregular row lengths and index distributions.
3. Briefly justify the final dataset in the report. A short summary table with rows, columns, nnz, and a one-line structural description is recommended.

Input vector. For every experiment, use a randomly generated dense vector of compatible size. You may keep the random seed fixed for reproducibility. Use Float32 as a data type. However, you can test other data type.

4. Implementation expectations

Implement a parser for Matrix Market files (.mtx) or reuse a minimal reader, as long as you document the choice.
Build at least one sparse representation for GPU execution. You may compare multiple representations when this helps answer a clear question.
Use the CPU implementation to validate numerical correctness of GPU results. State your validation criterion clearly, for example exact equality for integer-like cases or tolerance-based comparison for floating-point values (tolerance can be a user-parameter).
The CPU code is a baseline and reference, not the main optimization target. Do not spend effort on advanced CPU tuning that distracts from the GPU investigation.
An OpenMP baseline is optional but welcome if it is simple and clearly separated from the main GPU contribution.

5. Measurements to report

The minimum required metrics are: time-to-solution and computational throughput (FLOPS, typically reported as GFLOP/s). For SpMV, define clearly whether you use the standard estimate of $2 \cdot nnz$ floating-point operations per matrix-vector multiplication.

Runtime: report kernel time and clearly distinguish it from one-time setup costs such as file parsing or format conversion when relevant.
Throughput: report FLOPS/GFLOP/s consistently for all matrices and implementations.
Memory behavior: evaluating global-memory/cache miss behavior is encouraged. A coarse cache-miss study is acceptable even if your tooling investigation remains limited before the deadline.
Repetitions: use multiple runs and report either the average together with variability, or a justified stable statistic such as median/best-of-N.

6. What the analysis should explain

The report should interpret performance, not just present it. Your discussion should connect measured behavior to algorithmic technique and matrix characteristics.

Why does one sparse format or optimized kernel outperform another on a given subgroup of matrices?

How do row-length regularity, load imbalance, padding overhead, or irregular column accesses affect performance?
When does a format improve memory coalescing or locality, and when does it waste work or storage?
How close are your results to being memory-bound, and what evidence supports that interpretation?
Which observations are intrinsic to the matrix structure, and which are caused by implementation choices?

7. Recommended report organization (max 4 pages)

Section	What to include
Introduction	Problem statement, why SpMV matters, and the question your investigation addresses.
Methodology	Formats tested, CPU/GPU implementations, validation method, measurement methodology, and hardware/software environment.
Dataset	Short characterization of the 10 SuiteSparse matrices and the random dense vector input.
Results	Plots/tables for runtime and FLOPS/GFLOP/s; optional memory/cache indicators.
Discussion	Interpret the results in terms of format properties, matrix structure, and memory behavior.
Conclusion	Main lessons learned, limitations, and practical takeaways.

Practical interpretation: the strongest submissions will not simply say "kernel X is faster than kernel Y." They will explain under which matrix characteristics and memory-access conditions that statement is true, and when it stops being true. For example, what is the load balancing? Do you have too many access in global memory? Your SMs are not fully utilized. Etc... etc..

8. References and suggested reading

Add the following paper as a reference in the report, and use the additional papers below as strong optional reading when motivating format choice, load balancing, and GPU-specific optimization decisions.

Required added reference

1. Gao, J., Liu, B., Ji, W. and Huang, H., 2024. A systematic literature survey of sparse matrix-vector multiplication. arXiv preprint arXiv:2404.06047.
2. Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient Algorithm Design of Optimizing SpMV on GPU. In Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23), 2023. DOI: 10.1145/3588195.3593002.

Suggested papers from major HPC conferences

3. Bell, N., and Garland, M. Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors. In SC '09, 2009. *Comment: A classic starting point for COO, CSR-scalar, CSR-vector, and HYB trade-offs on GPUs.*

4. Ashari, A., Sedaghati, N., Eisenlohr, J., Parthasarathy, S., and Sadayappan, P. Fast Sparse Matrix-Vector Multiplication on GPUs for Graph Applications. In SC14, 2014. *Comment: Useful for irregular matrices and graph-oriented SpMV behavior.*

5. Greathouse, J. L., and Daga, M. Efficient Sparse Matrix-Vector Multiplication on GPUs Using the CSR Storage Format. In SC14, 2014. Comment: Important reference for CSR-Adaptive and structure-aware load balancing.
6. Liu, W., and Vinter, B. CSR5: An Efficient Storage Format for Cross-Platform Sparse Matrix-Vector Multiplication. In ICS '15, 2015. Recommended when discussing portability and format design beyond CSR/COO.
7. Merrill, D., and Garland, M. Merge-Based Parallel Sparse Matrix-Vector Multiplication. In SC16, 2016. Comment: Core reference for merge-based partitioning and balanced CSR execution.
8. Steinberger, M., Zayer, R., and Seidel, H.-P. Globally Homogeneous, Locally Adaptive Sparse Matrix-Vector Multiplication on the GPU. In ICS '17, 2017. Good reference for adaptive work distribution on GPUs.
9. Niu, Y., Lu, Z., Dong, M., Jin, Z., Liu, W., and Tan, G. TileSpMV: A Tiled Algorithm for Sparse Matrix-Vector Multiplication on GPUs. In IPDPS 2021. Useful when discussing local structure exploitation and tiled sparse formats.

How to use the literature in the report

Use one or two classical papers to define the baseline design space (for example Bell and Garland, or Greathouse and Daga).
Use one or two more recent papers to justify why modern GPU SpMV still depends strongly on sparsity pattern, load balance, and memory behavior.
Do not turn the literature review into a survey: cite only papers that help explain your own design and results.